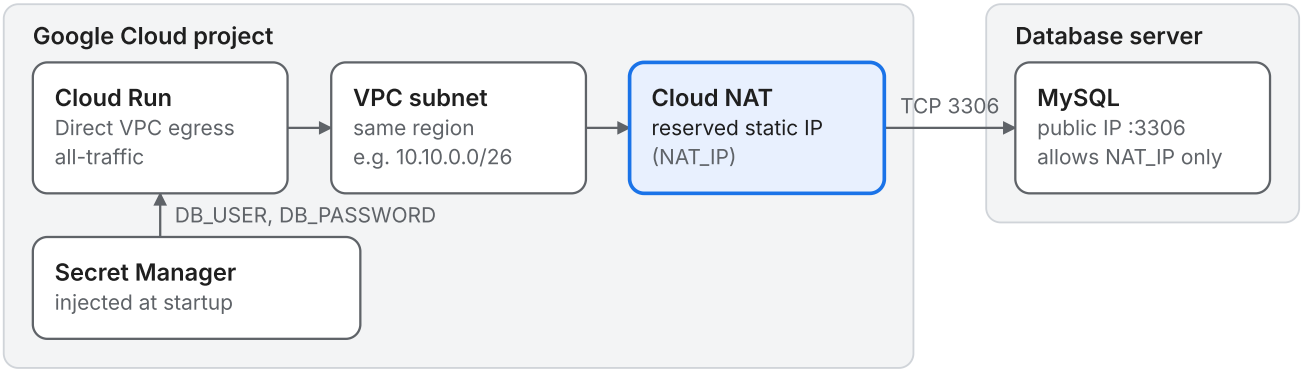
Cloud Run to external MySQL via Cloud NAT: setup guide

Overview

An external database usually allows connections only from known IPs. Cloud Run's default outbound IPs are shared and change, so you route its traffic through a VPC subnet and Cloud NAT with a reserved IP. The database then allows that one address, and Secret Manager supplies the credentials.

Every database connection leaves Google Cloud from one static IP



The database sees only NAT_IP; its firewall and the user@NAT_IP grant shut out every other address.

The setup has four parts, in this order: the NAT, the secrets, the deployment, then the database server. Commands use `$PROJECT`, `$REGION`, `$NET`, `$SUBNET` and `NAT_IP` as placeholders, and 203.0.113.10 as an example database IP.

1. Cloud NAT with a static IP

Cloud NAT gives every outbound connection from a subnet one source IP that you reserve. The NAT only serves the regions and subnets you attach it to, so it must be in the same region as the Cloud Run service.

```
# Variables used below
REGION=asia-south1; NET=egress-vpc; SUBNET=egress-subnet

# 1) VPC + subnet (a /26 is enough for Cloud Run Direct VPC egress)
gcloud compute networks create $NET --subnet-mode=custom
gcloud compute networks subnets create $SUBNET \
  --network=$NET --region=$REGION --range=10.10.0.0/26

# 2) Reserve the static external IP
gcloud compute addresses create egress-ip --region=$REGION
gcloud compute addresses describe egress-ip --region=$REGION --format='value(address)'

# 3) Cloud Router + Cloud NAT using only that IP
gcloud compute routers create egress-router --network=$NET --region=$REGION
gcloud compute routers nats create egress-nat \
  --router=egress-router --region=$REGION \
  --nat-custom-subnet-ip-ranges=$SUBNET \
  --nat-external-ip-pool=egress-ip
```

Flag	What it does
`--subnet-mode=custom`	No automatic subnets; you choose region and range
`--nat-external-ip-pool`	Use the reserved IP(s), not auto-allocated ones that can change
`--nat-custom-subnet-ip-ranges`	NAT only this subnet (use `--nat-all-subnet-ip-ranges` for all)
`--min-ports-per-vm`	Raise (e.g. 128+) if you see port exhaustion under heavy load

Check it with `gcloud compute routers nats describe egress-nat --router=egress-router --region=$REGION`. The NAT and the reserved IP are billed hourly, even when Cloud Run is idle.

---

2. Secret Manager for DB credentials

Store the DB username and password as secrets, give only the service's runtime account read access, and let Cloud Run inject them. Nothing sensitive goes in code, images or env files.

```
# Create (reads the value from stdin, so it never lands in shell history)
read -rsp 'DB password: ' P; echo
printf '%s' "$P" | gcloud secrets create db-password --replication-policy=automatic --data-file=-
printf '%s' 'appuser' | gcloud secrets create db-user --replication-policy=automatic --data-file=-

# Allow the Cloud Run runtime service account to read them
SA="$(gcloud projects describe $PROJECT --format='value(projectNumber)')-compute@developer.gserviceaccount.com"
gcloud secrets add-iam-policy-binding db-password \
  --member="serviceAccount:$SA" --role=roles/secretmanager.secretAccessor

# Rotate: add a new version (old versions stay until disabled/destroyed)
printf '%s' "$NEW" | gcloud secrets versions add db-password --data-file=-
gcloud secrets versions list db-password
gcloud secrets versions disable 1 --secret=db-password
```

Concept	Syntax / note
Mount as env var	`--set-secrets=DB_PASSWORD=db-password:latest`
Mount as file	`--set-secrets=/secrets/db-password=db-password:latest`
Pin a version	`db-password:3` instead of `:latest`
Pick up a new version	Env vars are read at startup, so deploy a new revision; file mounts with `:latest` refresh on read
Least privilege	Use a dedicated service account (`--service-account`) and grant `secretAccessor` per secret, not per project

---

3. Deploy to Cloud Run through the NAT

Attach the service to the subnet and send **all** egress through it. With the default `private-ranges-only`, calls to a public DB IP bypass the VPC and the NAT.

```
# Build the image with Cloud Build into Artifact Registry
gcloud artifacts repositories create apps --repository-format=docker --location=$REGION
IMAGE=$REGION-docker.pkg.dev/$PROJECT/apps/my-service:latest
gcloud builds submit --tag $IMAGE .

# Deploy with Direct VPC egress
gcloud run deploy my-service --image=$IMAGE --region=$REGION \
  --network=$NET --subnet=$SUBNET --vpc-egress=all-traffic \
  --set-env-vars=DB_HOST=203.0.113.10,DB_PORT=3306,DB_NAME=appdb \
  --set-secrets=DB_USER=db-user:latest,DB_PASSWORD=db-password:latest \
  --no-allow-unauthenticated

# Change one setting without rebuilding
gcloud run services update my-service --region=$REGION --update-env-vars=DB_NAME=otherdb
```

Option	Use it when
Direct VPC egress (`--network` , `--subnet`)	Default choice: no extra VMs, scales with the service
Serverless VPC Access connector (`--vpc-connector`)	Older setups, or features Direct VPC egress lacks; create with `gcloud compute networks vpc-access connectors create`
`--vpc-egress=all-traffic`	Always, for a NAT static IP to apply to public destinations
`--no-allow-unauthenticated`	Private API; callers need an identity token or `roles/run.invoker`

Spring Boot reads the injected variables in `application.yml` :

```
spring:
  datasource:
    url: jdbc:mysql://${DB_HOST}:${DB_PORT:3306}/${DB_NAME}?sslMode=REQUIRED&connectTimeout=10000
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 5          # Cloud Run instances x pool size must fit MySQL max_connections
      initialization-fail-timeout: -1  # start even if the DB is briefly down
```

---

4. MySQL server: listen, firewall, grant

The server must listen on its public address, let only the NAT IP reach port 3306, and have a user allowed only from that IP. `NAT_IP` below is the address from step 1.

```
# Listen on localhost + public IP (MySQL 8.0.13+ accepts a list).
# Use 0.0.0.0 if the public IP is not on the server's own interface (ip addr).
sed -i 's/^\s*bind-address\s*=.*/bind-address = 127.0.0.1,203.0.113.10/' /etc/mysql/mysql.conf.d/mysqld.cnf
systemctl restart mysql
ss -ltnp | grep 3306            # expect 203.0.113.10:3306 (or 0.0.0.0:3306)

# Firewall: only the NAT IP may reach 3306 (allow SSH before enabling!)
ufw allow OpenSSH
ufw allow from NAT_IP to any port 3306 proto tcp
ufw enable && ufw status
```

```
-- App user that can only log in from the NAT IP, with only the rights it needs
CREATE USER 'appuser'@'NAT_IP' IDENTIFIED BY '<strong password>';
GRANT SELECT, INSERT, UPDATE, DELETE ON appdb.* TO 'appuser'@'NAT_IP';
FLUSH PRIVILEGES;

-- Optional: require TLS for this user
ALTER USER 'appuser'@'NAT_IP' REQUIRE SSL;

-- Checks
SHOW VARIABLES WHERE Variable_name IN ('bind_address','port','require_secure_transport');
SHOW GRANTS FOR 'appuser'@'NAT_IP';
```

- The firewall rule and the `user@NAT_IP` host each limit access to the NAT address. Together, a leaked password can't be used from anywhere else.
- Hosted MySQL (cPanel or a cloud provider): add the NAT IP under Remote MySQL, or in the provider's allowlist or security group, instead of `ufw`.
- Lost admin access on Ubuntu: `sudo mysql --defaults-file=/etc/mysql/debian.cnf` logs in as the maintenance user.

---

5. Verify and troubleshoot

Call a DB health endpoint with an identity token, then read the real error in the logs if it fails.

```
URL=$(gcloud run services describe my-service --region=$REGION --format='value(status.url)')
curl -H "Authorization: Bearer $(gcloud auth print-identity-token)" "$URL/api/db/ping"

gcloud logging read 'resource.type="cloud_run_revision" AND resource.labels.service_name="my-service" AND textPa
yload:"Exception"' \
  --limit=3 --freshness=15m --format='value(textPayload)'

# On the DB server: does traffic from the NAT IP arrive?
timeout 30 tcpdump -ni any host NAT_IP and port 3306
```

Error in the logs	Cause	Fix
`Connection refused`	MySQL listens only on 127.0.0.1	Set `bind-address`, restart MySQL
`Communications link failure` or timeout, nothing in tcpdump	Firewall drops the NAT IP, or egress isn't `all-traffic`	Allow NAT IP on 3306 (server and provider firewall); redeploy with `--vpc-egress=all-traffic`
`Access denied for user 'u'@'NAT_IP' (using password: YES)`	No `user@NAT_IP`, or wrong secret value	Create or grant the user; add a new secret version and redeploy
`Access denied ... to database 'x'`	`DB_NAME` has no grant for that user	Fix `DB_NAME`, or `GRANT` on that database
`Permission denied on secret` at deploy	Runtime service account lacks `secretAccessor`	`gcloud secrets add-iam-policy-binding ...`
`403 Forbidden` in a browser	Service is private	Identity token, `gcloud run services proxy`, or public + app-level auth
Source IP isn't the NAT IP	NAT not covering the subnet, or wrong region	`gcloud compute routers nats describe ...`; NAT and service must share the region